Mohit Shakya

Backend Engineer & Independent Consultant — Java, Spring Boot, Apache Kafka

mohitshakya797@gmail.com | Gurgaon, India — remote worldwide
mohitshaky.github.io | github.com/mohitshaky | linkedin.com/in/mohit-shakya-9ab944110

PROFILE

Backend engineer with seven years building and operating production systems in telecom and banking — environments where an outage is a regulatory event and the audit trail has to survive questioning.

I work independently, taking on service builds, performance remediation and, increasingly, getting AI features into Java codebases that were never designed for them. Most engagements start with a diagnostic and end with the client's own team able to maintain what I built.

WHAT I DO

Build backend services
REST APIs, Kafka pipelines and BPMN-driven workflow services in Spring Boot. Delivered with tests, OpenAPI documentation, a container build and a runbook — handover is part of the work, not an afterthought.

Fix systems that have got slow
Kafka consumer lag that never recovers, p99 latency that has crept up across releases, database access nobody has revisited since it was written. I profile before changing anything and measure again after.

Make systems observable
Distributed tracing, metrics that map to user impact, and structured logging with correlation IDs, so the next incident is diagnosed from a dashboard rather than by reading log files.

Integrate AI into existing Java estates
Retrieval over a client's own documents using Spring AI or LangChain4j, with citations, grounding guardrails, prompt versioning and a token cost ceiling — the controls a compliance review asks for.

SELECTED ENGAGEMENTS

Client names withheld under confidentiality. Described by problem and approach.

> **TELECOM — ORDER MANAGEMENT PLATFORM**
>
> **Order pipeline that backed up under campaign load**
>
> **Problem.** A pipeline carrying hundreds of thousands of orders a day processed reliably at baseline volume but ran up to an hour behind whenever a marketing campaign launched, leaving orders in intermediate states for support to resolve by hand.
>
> **Approach.** Traced the pipeline to locate the real queueing point, redesigned consumer groups and partitioning around the correct partition key, moved blocking downstream calls off the poll loop, and introduced a retry topic with dead-letter handling so a single malformed message could no longer stall a partition.
>
> **Outcome.** Campaign peaks absorbed without operator intervention; stuck orders became an alertable condition rather than a customer complaint.

> **BANKING — ACCOUNT EVENT PROCESSING**
>
> **Audit trail that could not be shown to be complete**
>
> **Problem.** Account movements were published as events, but answering one compliance question cost a few hours of engineering time, querying three separate stores and reconciling by hand, with no assurance the result was correct.
>
> **Approach.** Established idempotent consumers keyed on a stable event identifier, made the write path append-only, and separated the audit projection from the operational read model so either could be rebuilt without disturbing the other.
>
> **Outcome.** History reconstructible from the event log alone; a single projection query replaced the manual reconciliation.

> **TELECOM — SUBSCRIPTION LIFECYCLE**
>
> **One business rule change meant releasing six services**
>
> **Problem.** Subscribe, upgrade, suspend and cancel logic had accumulated across several services over years. The true sequence existed only as tribal knowledge.
>
> **Approach.** Modelled the lifecycle explicitly as a BPMN process so the sequence became a reviewable artefact, kept service integration event-driven, and made each transition individually traceable.
>
> **Outcome.** Rule changes became a process-definition edit rather than a coordinated multi-service release.

PUBLIC CODE

`enterprise-rag-service`	Retrieval-augmented generation built as a production service. Spring AI over pgvector, multi-tenant isolation proved by integration test, a grounding policy that refuses rather than fabricates, versioned prompts and per-tenant token budgets.
`order-lifecycle-bpmn`	Flowable BPMN orchestration with asynchronous Kafka provisioning and multi-tenant partitioning. The process definition is the source of truth.
`banking-account-service`	Append-only audit projection rebuildable from the event log; idempotent consumers under at-least-once delivery.
`offer-promo-engine`	Latency-sensitive rule evaluation with a deliberate Redis caching boundary, OAuth2 resource server and Kubernetes manifests.

TECHNICAL

LANGUAGES	Java 17 / 21, SQL, JavaScript	**FRAMEWORKS**	Spring Boot 3, Spring Security, Spring Data, Spring AI, Flowable BPMN
MESSAGING	Apache Kafka — consumer groups, partitioning, retry and dead-letter topics, delivery semantics	**DATA**	PostgreSQL, pgvector, MongoDB, MySQL, Redis, Flyway
AI	Spring AI, LangChain4j, RAG, vector search, evaluation harnesses	**OPERATIONS**	Docker, Kubernetes, Prometheus, Grafana, OpenTelemetry, GitHub Actions
TESTING	JUnit 5, Mockito, Testcontainers, integration and contract testing	**BUILD**	Gradle, Maven

HOW I ENGAGE

Fixed price against a written scope, with engineering hours stated, agreed before work starts. Full IP assignment on final payment. First milestone carries no risk: if the deliverable is not what we agreed, it is not billed and the code is yours regardless.

Typical engagements: a five-day Backend Health Check, a two-week Fix & Harden, a three-to-four week service or pipeline build, or a fractional backend lead arrangement at ten hours a week. I sign GDPR Article 28 DPAs and EU Standard Contractual Clauses; W-8BEN on file for US clients; exports of services from India are zero-rated, so no GST or VAT is added to an invoice.